

PlateSense

Project Introduction

Nowadays, capturing objects from images or videos is a very common task as many examples of image recognition explain how to find cars, cats, dogs, humans... Google, Amazon, and many others provide API, libraries and models, and many researchers continue to give better and better algorithms, mathematical progress and ideas to enhance that activity.

So, checking if a given image is about food is easy.

But what if you have to guess what the food sitting on the table **actually is** ? And what about finding every dish, dessert, entries, bread and drink that is on the table?

What is this project about

This project is an application of machine learning to the task of identifying and recognizing which food items are present on a plate. We put together a system that looks at food images and which in turn is able to very accurately identify, classify and label each item.

Why is this project Developed

The aim of this project is to present a better, smarter, and more efficient food recognition which in turn will see use in diet and health, we also look at how it can improve patient care in the hospital setting with meal tracking. Also we see this as a way to reduce human error and to put AI in the day to day analysis of food.

When is this project is used

This technology is applied in which we do not have manual input for instance in diet tracking apps, smart restaurant systems, and health care monitoring tools. Also it is very much so when speed and accuracy of food identification is of the essence which is the case when we are to determine what food is put forth or what has been consumed.

Flow of the Project

The initial approach into using HAAR like features (what we think of as traditional computer vision) did see success with simple items like plates and bottles, however we did see the algorithm fail when it came to very diverse food stuffs like risotto, salad, or spaghetti.

- HAAR uses simple rigid features that are not present in complex food surfaces.

To improve on that we had to alternatively use Convolutional Neural Networks (CNNs).

CNNs do well at identifying the fine detailed visual elements of food (texture, color, content) outperformed by no other model; also they work best when the identified food item is the primary focus of the image.

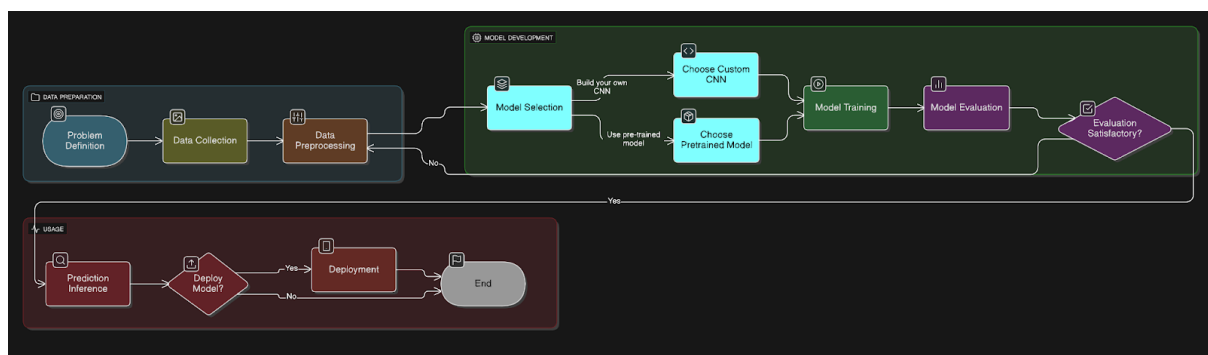
- This step was in which we went from basic shape identification to the recognition of content.

But what is the problem in the CNN is

- “Basic” CNN is able to say “ok, that’s a cat” if you give an image with a cat as the main object of the scene. But when you give it an image with several objects in a complex scene containing cats, dogs, plants, and so on in a garden, the CNN will say “ok... 3% of cat in the image, 10% dogs, maybe a pizza there (0.3%)”.
- CNN hasn’t checked regions of the image, it watched the entire image and tried to evaluate the scene. Actually, it is right, there are actually 3% of cats in the entire image.

Conclusion that the CNN: That if the one item present per plate use CNN

This is the First approach



Step 1: Data Collection

Gather images for each food category.

Sample datasets:

- <https://datasetninja.com/food-recognition#download>
- <https://www.kaggle.com/datasets/trolukovich/food11-image-dataset>
- <https://www.kaggle.com/datasets/rkuo2000/uecfood256>

Step 2: Image Preprocessing

- Resize all images to the same size (e.g., 224×224 pixels).
- Normalize pixel values (divide by 255).

- Split the dataset into:
 - Training set (70%)
 - Validation set (20%)
 - Testing set (10%)
- Apply data augmentation (rotation, flipping, zoom) to avoid overfitting.

Step 3: Model Selection

- Choose a CNN-based model.
- Build your own CNN
- Use a pre-trained model (Transfer Learning):
ResNet, VGG16, or MobileNet — already trained on ImageNet.

Step 4: Model Training

- Feed training images into the model.
- The CNN learns visual patterns (color, texture, shape) of each food item.
- Adjust parameters like **learning rate**, **batch size**, and **epochs**.
- Monitor training accuracy and loss using the validation set.

Step 5: Model Evaluation

- Test the model using the test dataset (unseen images).
- Calculate metrics:
 - **Accuracy**
 - **Precision & Recall**
 - **Confusion Matrix**

Step 6: Prediction / Inference

- Give a new image of food to the trained model.
- The model outputs a label like:
“Predicted: Salad” or “Predicted: Burger.”

This is the method used only for detecting one image per plate using CNN that is the main drawback of this model

CNN hasn't checked regions of the image, it watched the entire image and tried to evaluate the scene. Actually, it is right, there are actually 3% of cats in the entire image.

(This is the approach is suitable for the requirement)

Use R-CNN, YOLO, Fast and Faster R-CNN, and so on

Step 1: What is the problem statement

- Input: Identify and locate multiple food items in a single plate image.
- output : Plate with [rice,veggies and chicken]

Step 2: Data Collection

Gather images for each food category.

Sample datasets:

- <https://datasetninja.com/food-recognition#download>
- <https://www.kaggle.com/datasets/trolukovich/food11-image-dataset>
- <https://www.kaggle.com/datasets/rkuo2000/uecfood256>

These dataset have labelling if the dataset do not have labelling we need to manually make the labelling

Use **annotation tools** like:

- **LabelImg** → for bounding boxes
- **Roboflow** → easy online labeling and format conversion
- **CVAT** → for team-based labeling

Draw rectangles (bounding boxes) around each food item and assign the correct label (e.g., “Rice,” “Curry,” “Salad”).



Step 3: Image Preprocessing

- Convert all annotations to the correct format for your model (YOLO or COCO format).
- Split your data:
 - **Training** → 70%
 - **Validation** → 20%
 - **Testing** → 10%
- Resize all images (e.g., 640×640 for YOLO).

Model Selection

- Now choose the object detection model.
- YOLO (You Only Look Once)
 - Fast, accurate, and works well even with small datasets.
 - Variants: **YOLOv5**, **YOLOv8**, **YOLO-NAS** (latest).
- **Other options:**
- **Faster R-CNN** – high accuracy but slower.

Model Training

- Load your dataset (images + labels).
- Set training parameters:
 - Epochs (e.g., 50–100)
 - Image size (e.g., 640×640)
 - Batch size (e.g., 16)
- Train using GPU for faster performance.
- The model learns to detect and label each food item.

Model Evaluation

Test the model using unseen images.

Check metrics like:

- **mAP (mean Average Precision)**
- **Precision & Recall**

Prediction / Output

Use the trained model to detect foods in a new image.

The output:

- Boxes drawn around food items
- Labels (like "Rice", "Curry", "Salad")
- Confidence scores (e.g., 0.92, meaning 92% sure)